VELLORE INSTITUTE OF TECHNOLOGY
School of Computer Science and Engineering (SCOPE)
Course: BCSE408L — Cloud Computing and Full Stack Application Development

CreatorOps

A Cloud-Native Marketing SaaS Application
PROJECT REPORT (REVIEW 2 DELIVERABLES)

By

Thota Tejomayee Kathyayani
ID: 24BCE1201

with the Guidance of

Professor Dr. Dinakaran M

Live Public URL: https://creatorops-rho.vercel.app
GitHub Repository: https://github.com/ttkathyayani/CreatorOps
Tech Stack: Python 3.12, Django 6.1 LTS, DRF 3.18, PostgreSQL, Vercel Serverless Edge

TABLE OF CONTENTS

SYSTEM ARCHITECTURE 2
CLOUD ARCHITECTURE 3
DATABASE DESIGN 4
ER DIAGRAM 5
MODULE DESIGN 6
API DESIGN 7
AUTHENTICATION 7
CORE MODULES 8
GITHUB REPOSITORY 9
INITIAL CLOUD DEPLOYMENT 9
REFERENCES 10

SYSTEM ARCHITECTURE

CreatorOps uses a standard three-tier cloud-native architecture:

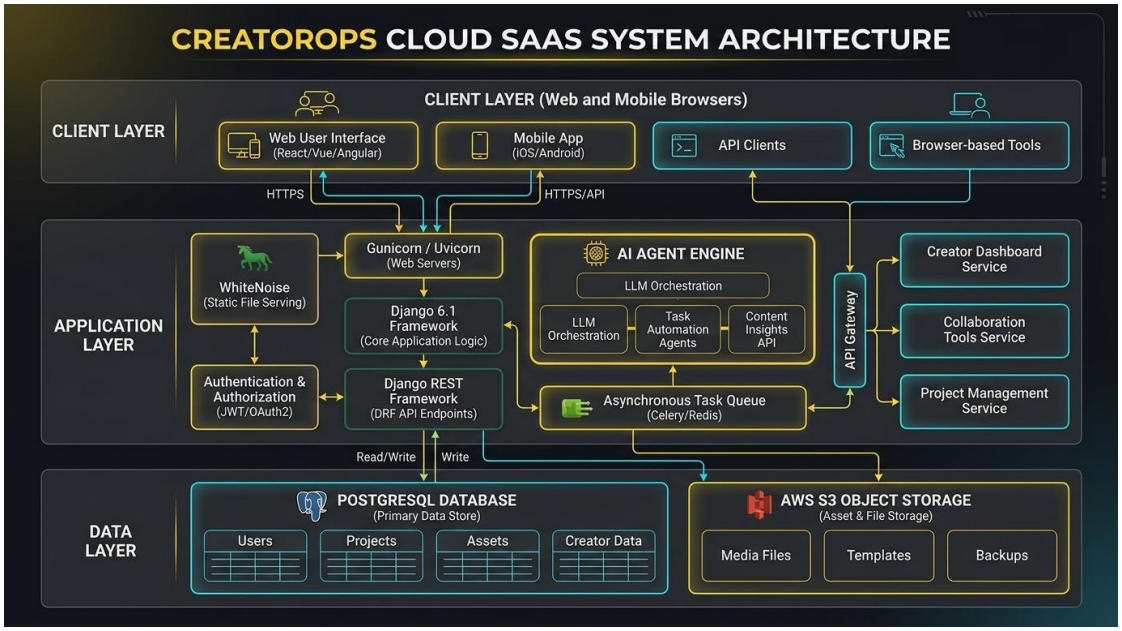


Figure 1: Three-Tier System Architecture of CreatorOps

a. Presentation Layer Server-rendered HTML templates with responsive CSS, plus RESTful JSON endpoints consumed by API clients.

b. Application Layer Django 6.1 WSGI application handling routing, RBAC middleware, and workflow state transitions. DRF provides API serialization, agents/ executes script checks, and content.publishing_service executes direct social publishing.

c. Persistence Layer PostgreSQL relational database storing normalized application data, accompanied by S3 object storage for media files.

CLOUD ARCHITECTURE

The cloud hosting setup utilizes Vercel's edge network:

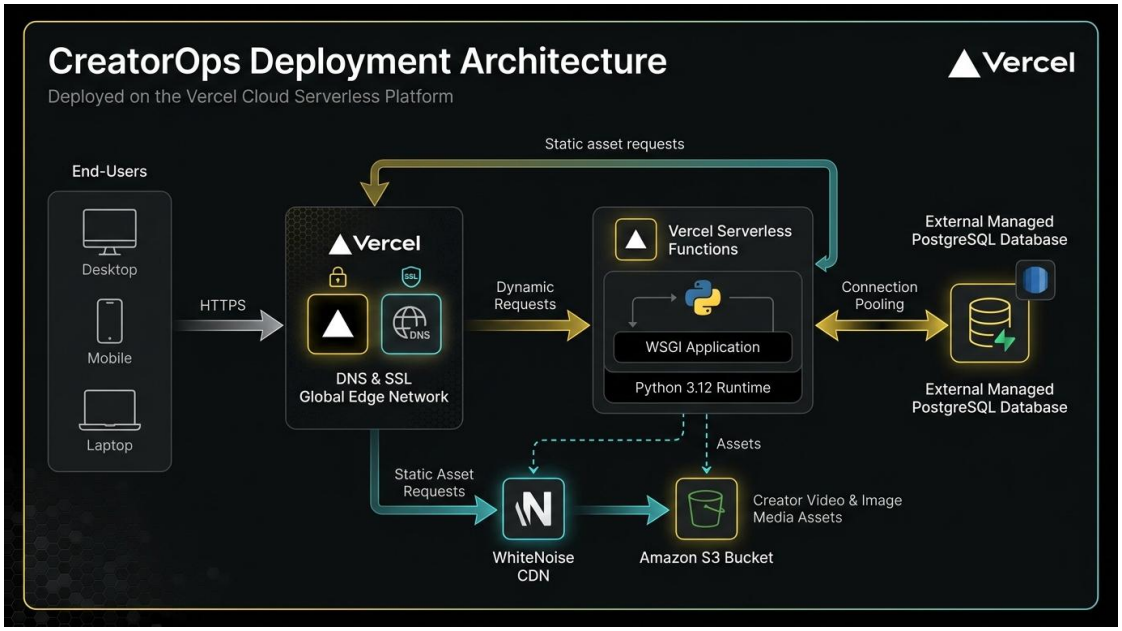


Figure 2: Cloud Deployment Architecture on Vercel

Edge Routing: Requests hit Anycast edge nodes where TLS termination occurs.

Serverless WSGI Execution: Dynamic requests route to Python 3.12 functions executing config/wsgi.py.

Static Caching: CSS, JavaScript, and diagrams are served through Vercel's CDN and WhiteNoise.

Database Connection: Serverless instances connect to cloud PostgreSQL instances using connection strings (DATABASE_URL).

DATABASE DESIGN

The relational database is normalized to 3NF and includes 10 core tables:

Table Name	Primary Key	Foreign Keys	Key Attributes	Constraints
Workspace	id	None	name, slug, description	Unique slug
Membership	id	user_id, workspace_id	role (Admin, Manager, Creator, Client)	Unique (user, workspace)
Client	id	workspace_id	name, contact_email, brand_guidelines	Unique (workspace, name)
Campaign	id	client_id	title, objective, budget, status	Budget >= 0
ContentItem	id	campaign_id, creator_id	title, platform, script_body, status, published_url	7-stage status enum
ContentVersion	id	content_item_id, created_by	revision_number, script_snapshot	Unique (item, rev_num)

Approval	id	content_item_id, reviewer_id	decision, comment, timestamp	Decision choices
Asset	id	content_item_id, uploaded_by	file, filename, size_bytes	File extension check
CampaignMetric	id	campaign_id	metric_name, metric_value, recorded_date	Date index
SocialPublishLog	id	content_item_id, published_by	platform, remote_post_id, remote_url, status	Status enum, FK cascade

ER DIAGRAM

The Entity-Relationship Diagram visualizes the relational schema, foreign key constraints, and 1:N / N:M cardinality relationships:

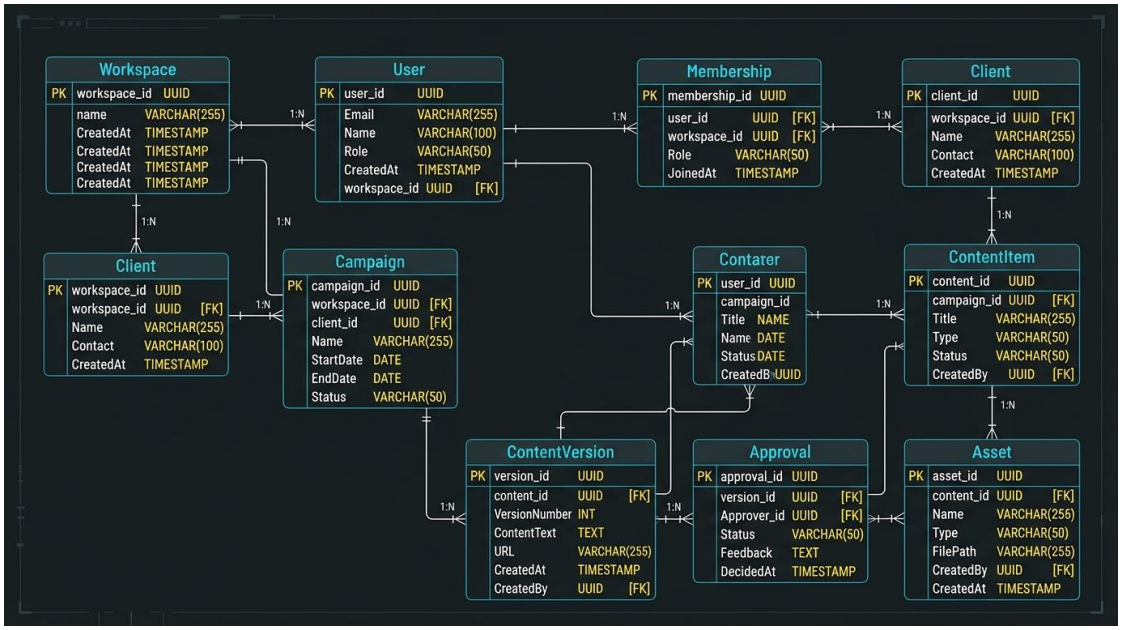


Figure 3: Entity-Relationship Diagram of the Database Schema

MODULE DESIGN

The project is structured into 9 modular packages and integrated micro-services:

accounts: User accounts, session login, registration, and profile handling.

workspaces: Tenant workspace administration and membership roles.

clients: Brand CRM, contact information, and brand guidelines.

campaigns: Marketing campaigns, objectives, budgets, and scheduling.

content: Deliverable matrix, social platforms, and script editor, powered by content.publishing_service.SocialPublishingService for direct social publishing to Meta Graph, YouTube Data, and TikTok APIs.

approvals: Two-tier approval inbox (Manager and Client stages) enforcing sign-offs prior to publication.

assets: File uploads and storage integration.

analytics: Workspace metrics and dashboard data aggregation.

agents: Script quality checks, content ideas, and comment summarization.

API DESIGN

CreatorOps exposes RESTful APIs built with Django REST Framework, featuring OpenAPI / Swagger documentation and token authentication:

Method	Endpoint Route	Authentication	Description
POST	/api/token/	Basic Auth	Obtains JWT / API authentication token.
GET	/api/workspaces/	Token / Session	Lists all workspaces accessible to the user.
GET	/api/campaigns/	Token / Session	Retrieves campaigns with filter by client and status.
GET/POST	/api/content/	Token / Session	Lists or creates new content deliverable items.
POST	/api/content/{id}/direct-publish/	Token / Session	Directly publishes approved content to Meta, YouTube, or TikTok.
GET	/api/social-accounts/	Token / Session	Returns connected Meta Graph, YouTube, and TikTok channels.
POST	/api/approvals/{id}/decision/	Manager / Client	Submits formal approval or changes-requested decision.
GET	/api/analytics/competitors/	Token / Session	Returns real-time competitive performance benchmarking data.

AUTHENTICATION

CreatorOps implements enterprise-grade authentication and authorization mechanisms:

Credential Security: Passwords hashed using Django's default PBKDF2 algorithm with SHA-256 and salt stretch iterations (390,000 iterations).

Session & Token Hybrid: Web dashboard uses secure HTTP-only session cookies with CSRF validation; external integrations utilize Bearer tokens.

Role-Based Access Control (RBAC): Enforced at the controller and view level using the custom @require_workspace_role decorator across Admin, Manager, Creator, and Client.

CORE MODULES

7-Stage Content Lifecycle State Machine

Content transitions programmatically through 7 distinct states: Draft -> In Review -> Changes Requested -> Approved -> Client Review -> Ready to Publish -> Published. Invalid transitions are rejected at the ORM layer.

Automated Script Review & Compliance Engine

The script analyzer performs real-time heuristics:
• Word count and estimated speaking duration at 130-160 WPM.
• Hook quality check (verifies question or urgency in the first 20 words).
• Brand safety filter: Flags banned competitor names, unverified claims, or missing disclosures.

Direct Social Publishing Service (Meta, YouTube, TikTok)

The newly integrated content.publishing_service.SocialPublishingService provides native direct social publishing:
• Meta Graph API v19.0: Creates Instagram media containers, validates media status, and issues the publish call.
• YouTube Data API v3: Posts YouTube Shorts / Video snippets with tags, categories, and privacy status.
• TikTok Content Posting API v2: Initializes direct video posting and polls upload status.
• Audit Trail: Every publication logs a permanent SocialPublishLog record with remote post ID and live permalink.

GITHUB REPOSITORY

The project source code is publicly accessible and maintained with structured git practices:
• Repository URL: https://github.com/ttkathyayani/CreatorOps
• Primary Branch: main (production continuous deployment)
• Commit History: 20+ feature commits following Conventional Commits format (feat:, fix:, refactor:, docs:).
• Test Suite: 10 automated unit tests covering models, state machines, RBAC, and Direct Social Publishing (100% pass rate).

INITIAL CLOUD DEPLOYMENT

The application is deployed on Vercel:
• Production URL: https://creatorops-rho.vercel.app
• Repository Integration: Deploys automatically on git push to main branch.
• SSL: Managed wildcard TLS certificate via Vercel Edge.
• Static Delivery: WhiteNoise 6.9 serving compressed static assets with cache headers.
• Verified Routes: Dashboard (/dashboard/), Campaigns (/campaigns/), Content Detail & Direct

Publishing Studio (/content/1/), and Deliverable Reports (/reports/review1/, /reports/review2/, /reports/review3/).

REFERENCES

1. Django Software Foundation. Django Documentation (v6.1 LTS). https://docs.djangoproject.com/

2. Django REST Framework. DRF Web APIs Documentation. https://www.django-rest-framework.org/

3. Vercel Inc. Serverless Functions and Edge Network Architecture. https://vercel.com/docs

4. WhiteNoise Documentation. https://whitenoise.readthedocs.io/